

Project Overview

Entry-point document for the storage capacity management and deviation detection project. Read this first; it links to the rest of the documentation set.

1. Summary

This project delivers an AI / agentic system (built on LangGraph) for **storage capacity management and deviation detection** across a **heterogeneous, multi-OS fleet of at least 500 servers** (AIX, all Linux distributions, and Windows Server). It collects non-sensitive storage metadata from every host, compares each server against a per-OS reference standard ("golden image"), detects deviations from the expected layout, forecasts filesystem saturation in both bytes and inodes, and proposes prioritized remediations. Deterministic rules perform all measurement and detection so that no number is ever hallucinated; an LLM layer only explains, prioritizes, and recommends; and a human-in-the-loop (HITL) step validates low-confidence or high-impact findings. The project is currently in the **scoping / specification phase**: there is no application code yet, and this repository holds the documentation and structured-data artifacts needed to start cleanly.

2. Purpose & business context

Large server fleets accumulate storage problems that are invisible until they become incidents. This project exists to address four recurring, costly pain points:

- **Full-filesystem outages.** A filesystem that silently fills up — on either bytes or inodes — takes down the service that depends on it. These incidents are predictable in hindsight but are not caught today because there is no continuous, fleet-wide view of how fast each filesystem is approaching its effective ceiling. Detecting saturation ahead of time turns a 2 a.m. incident into a routine extension.
- **Wasted capacity.** Volumes are over-provisioned, obsolete data is never reclaimed, and large allocations (old backups, core dumps, dumps) sit unnoticed. Capacity is bought to cover slack that already exists in the fleet.
- **Misplaced data.** Application and customer data ends up on system partitions, which both wastes the partitions sized for the OS and creates reliability and compliance exposure. Today there is no systematic way to quantify "data on a system partition" without reading file contents.
- **Compliance drift.** Mount options and partition separation drift away from CIS baselines (for example `/tmp` without `noexec`, or `/var/log` not on a separate filesystem). Without a reference to compare against, drift goes undetected.

The common root cause is the absence of a consistent, machine-comparable model of what each server's storage *should* look like for its OS and version, and of a continuous record of what it *actually* looks like over time. This project supplies both: a single collection schema and a set of per-OS reference standards, feeding a pipeline that measures, compares, forecasts, and recommends.

3. The five objectives

These objective IDs and meanings are canonical and are used identically across the whole documentation set.

- **01 – Deviation detection vs standards.** Spot servers whose partitioning / filesystem layout differs from the model expected for their OS and version.
- **02 – Optimization (useless / obsolete / misplaced).** Reclaim space — obsolete data, over-provisioned volumes, and application data sitting on system partitions.
- **03 – Reliability & full-filesystem prevention.** Detect ahead of time the filesystems about to fill up (in **bytes AND inodes**) to avoid incidents.
- **04 – Operational efficiency & forecasting.** Standardize storage structures and anticipate growth and capacity needs.
- **05 – Risk reduction & compliance.** Ensure only appropriate data resides on system partitions; check mount options and ACLs against CIS baselines.

4. Scope

Operating systems (all first-class, delivered together)

Every OS family below is treated **as a first-class citizen** and is delivered together in a single release.

OS family	Grouping / detail	Default filesystem	First-class?
AIX	LVM (rootvg / datavg)	JFS2	Yes
RHEL / Oracle Linux / Rocky / AlmaLinux	XFS by default, LVM common	XFS	Yes
Debian / Ubuntu	ext4 by default	ext4	Yes
SUSE / SLES	btrfs by default (subvolumes + snapshots)	btrfs	Yes
Windows Server	NTFS / ReFS, Storage Spaces	NTFS / ReFS	Yes

Fleet size, data sensitivity, and processing environment

- **Fleet size:** at least **500 servers**. This threshold ensures statistically meaningful coverage of the fleet and robust, fleet-wide deviation detection.
- **Non-sensitive metadata only:** the system collects capacities, topology, mount options, and a non-sensitive ACL summary on Windows. It never collects filenames, file owners, file contents, or named ACEs/SIDs. Privacy concerns are minimized by design because only metadata is gathered.
- **Processing environment:** analysis can run **locally, in a lab, or in the cloud (GCP / Azure)**, depending on scalability and integration needs. The collection schema and pipeline are independent of where processing runs.

5. Approach & method

The method rests on a strict division of labor and a continuous record of the fleet over time.

- **Rules do all measurement and detection.** Every number — capacity, used bytes, free bytes, inode counts, growth rate, projected days to full, deviation, severity — is produced by deterministic rules. This guarantees zero hallucination on any figure.
- **The LLM only explains, prioritizes, and recommends.** Once the rules have measured and detected, the LLM layer triages findings, writes human-readable explanations, prioritizes by impact, and proposes remediation text. It never invents or alters measurements.
- **Human-in-the-loop (HITL).** Low-confidence or high-impact findings are routed to the relevant teams for validation before they are acted on. Confirmed / false-positive verdicts feed back to improve future runs.
- **Repeated UTC collection (time series).** Collection is repeated over time and stored **append-only** — never overwriting prior runs. All timestamps are UTC ISO-8601. This time series is what makes forecasting possible: a single snapshot cannot project growth, whereas a sequence of daily points can.
- **Reference "golden image" comparison.** Each host is compared against the reference standard for its OS and version. Hosts that have no matching reference are explicitly routed to human validation rather than being silently declared "compliant" — this avoids silent false negatives, which would otherwise be the largest blind spot in the system.

6. End-to-end process steps

1. **Collect** filesystem and storage metadata from each host.
2. **Normalize / standardize naming** — apply canonical forms and resolve account-specific naming conventions so that records are comparable and joinable to reference standards.
3. **Classify** systems and data (for example, deriving system vs data storage).
4. **Compare placement to the reference** — match each host's actual filesystem layout against its golden image.
5. **Detect deviations** and anomalies.
6. **Prioritize & recommend** — rank findings and generate actionable recommendations.
7. **HITL validation** — validate low-confidence or high-impact findings with the relevant teams.

7. Deliverables

- **Deviation reports with severity** — per-host and fleet-wide reports classifying each deviation by severity.
- **Actionable remediations** — concrete recommendations (extend a volume, migrate misplaced data, reclaim obsolete data) tied to specific findings.
- **Dashboards** — aggregated insights and metrics for capacity-management and operations teams.
- **Structured data** — the collected metadata and derived results in structured form for downstream reporting and analytics.

8. Stakeholders

- **Account management** — owns the relationship and validates findings against account-specific context.
- **Platform architects** — define and own the reference standards and the target storage structures.
- **Technical leads** — act on remediations and operate the affected systems.
- **Capacity-management teams** — consume forecasts and dashboards to plan capacity and prevent incidents.

9. Key locked decisions (canonical data model)

The following decisions are locked and are matched by every document in the set.

- **Delivery unit = ONE envelope file per collection run.** Each run produces a single envelope object that carries every server in the fleet:

```
{
  "snapshot_id": "string",    // REQ – collection-run identifier (RUN level)
  "generated_at": "string",   // REQ – UTC ISO-8601, file generation time (RUN level)
  "host_count": number,      // REQ – number of elements in hosts[] (RUN level)
  "hosts": [ /* ONE document per server – host schema below */ ]
}
```

- `snapshot_id`, `generated_at`, and `host_count` live at the **envelope / run level**.
- Each `hosts[]` element is one server's document. It carries its own `scan_timestamp` (UTC ISO-8601, that host's scan time) but does **not** carry its own `snapshot_id` – the run identifier belongs to the envelope.
- For very large fleets, **JSONL** is accepted: one host object per line, where each line then carries both `snapshot_id` and `scan_timestamp` so that lines remain attributable to a run.
- **One OS-agnostic per-host schema** – identical across AIX, Linux, and Windows. It contains:
 - Identity & collection metadata: `host_id`, `hostname`, `gsma_code`, `host_role_env`, `platform` (`aix` | `linux` | `windows`), `os_family`, `os_distribution`, `os_version`, `architecture`, `scan_timestamp`, `collection_method`, `scan_confidence`.
 - `filesystems[]` – the **universal denominator**: every mountable Unix filesystem and every Windows volume (drive letters and folder mount points). This is the only structure mandatory on all OS.
 - `storage_topology { groups[], volumes[], disks[] }` – neutral levels with a `*_kind` discriminator (`group_kind = lvm-vg | storage-pool`; `volume_kind = lvm-lv | partition | virtual-disk`; and so on). Empty for plain-partition Linux or basic-disk Windows.
 - `paging[]` – swap / paging space / pagefile.
- **Linkage by keys, not nesting.** A filesystem references its underlying volume/group via `backing_ref` / `group_ref`. This weak coupling lets the pipeline derive `storage_class` (system vs data) generally – AIX rootvg/datavg, Linux system/data VGs, Windows C: system vs other data drives.
- **Sizes are EXACT BYTES** (integers) everywhere – never blocks, KB, MB, GB, or percentages. Native KB / blocks (`df -k`, `lsps`) are normalized to bytes at collection time. Rounded-GB sources are rejected at ingestion.
- **Conditional / nullable per OS+fstype = explicit null**. A field that does not apply to a given OS or fstype is returned as explicit `null` (verified N/A), **never omitted**, so that "N/A" is distinguishable from "not collected". The collection command branches per platform; the schema does not.
- **Append-only UTC time series.** Repeated collection over time is appended, never overwritten – this is what enables forecasting.

- **Non-sensitive metadata only.** Capacities, topology, mount options, and a non-sensitive ACL summary on Windows. No filenames, no owners, no file contents, no named ACEs/SIDs.
- **storage_class and all derived metrics are computed downstream.** `storage_class` (system vs data), percentages, growth rates, deviations, and severities are **derived by the pipeline** — they are not collected.
- **Rules vs LLM.** Rules measure and detect; the LLM explains, prioritizes, and recommends; HITL validates low-confidence or high-impact findings.

10. In scope now vs next steps

In scope now — this documentation set:

- Project overview (this document)
- [Business goals](#)
- [Requirements](#) (functional and non-functional)
- [Risk register](#)
- [Data collection spec](#) (the OS-agnostic storage collection schema)
- [Data-team request](#) (the message to the data team)

Next steps (explicit, upcoming — not detailed in this set):

- **Reference standards / golden images** — the per-OS, per-version reference layouts to compare against.
- **OS-by-OS field-applicability matrix** — which fields are collected, nullable-N/A, or different, by OS and fstype.
- **Field-by-field rationale + Ansible collection** — why each field exists and how it is collected per OS (AIX, Linux, Windows).
- **LangGraph architecture** — the pipeline (state, nodes, branches) that processes the collected data.
- **Machine-validatable JSON Schema file** — a JSON Schema encoding the storage schema (and applicability) as a machine-verifiable contract for the data team.

11. Glossary

- **rootvg / datavg** — AIX LVM volume groups. `rootvg` holds the operating system; `datavg` holds application/data volumes. Keeping data on `datavg` (separate from `rootvg`) is the central "correct placement" signal on AIX, generalized to `storage_class` on all OS.
- **VG / LV / PV** — LVM building blocks. A **Volume Group (VG)** pools storage; a **Logical Volume (LV)** is a carved-out volume on which a filesystem lives; a **Physical Volume (PV)** is a disk/device that backs a VG.
- **storage_pool / virtual-disk** — the Windows Storage Spaces equivalents of a group and a volume; modeled by the neutral `group_kind = storage-pool` and `volume_kind = virtual-disk` discriminators.
- **storage_class** — a derived classification of a filesystem as **system** vs **data**, computed by the pipeline from the backing volume/group. Generalizes AIX rootvg/datavg, Linux system/data VGs, and Windows C: vs other drives across all OS.
- **snapshot vs scan** — a **snapshot** (`snapshot_id`) is one complete collection *run* across the fleet (envelope level); a **scan** (`scan_timestamp`) is one individual host's collection within that run (host level).
- **golden image** — the per-OS, per-version reference standard for the expected storage layout, against which each host is compared.
- **gsma_code** — an account/site identifier carried per host in the identity metadata.

- **HITL (human-in-the-loop)** — the validation step where low-confidence or high-impact findings are reviewed by people before being acted on.
- **inode** — a filesystem object-table entry. A filesystem can run out of inodes even with free bytes (many small files), so saturation is tracked in both bytes and inodes (O3).
- **btrfs subvolume** — an independently manageable, snapshottable namespace within a btrfs filesystem; the default layout on SUSE / SLES.

12. Status

Scoping / pre-implementation. No application code yet; this repository holds documentation and structured-data artifacts to start cleanly. Dated **2026-06-15**.